DEV WEEKEND CHALLENGE · DOG DAYS EDITION

TELLTAIL

A dog cannot tell you where it hurts.
Ten million rows of collar data can.

Complete build specification and AI prompt pack

Target categories: **Best use of Snowflake** + **Best use of Solana**

Submissions due Monday 17 August 2026, 12:29 PM IST

Author: Soumyadeep Dey · github.com/SoumyaEXE

Contents

1. The pitch in one page

Every consumer dog tracker on the market computes activity minutes and step counts, because that is what you get from `AVG()` and `SUM()`. But a vet does not diagnose from an average. A vet diagnoses from a **sequence**: this, then that, then this again, in that order, that many times.

TELLTAIL streams 10.6 million rows of real dog collar telemetry into Snowflake, classifies every second into a behavioural state, and then hunts for **clinical syndromes expressed as regular expressions over rows** using `MATCH_RECOGNIZE`. Not "scratching is high today." Instead: head shake, scratch cluster, head shake, scratch cluster, emerging from a rest state, which is otitis and nothing else.

> **The thesis, stated the way FERVOR stated its own:** a threshold cannot detect a syndrome. Aggregation tells you a number changed. Only row pattern recognition tells you that events happened in a diagnostic order. Snowflake is the only warehouse where a differential diagnosis is a SQL clause, and the theme is not decoration here: the entire reason this system must exist is that the patient cannot speak.

What each platform does that the other cannot

```
Kaggle CSV --replay--> Snowflake --ML--> Snowflake --pattern--> Snowflake --attest--> Solana
10.6M rows @ 100Hz     VARIANT land      CLASSIFICATION      MATCH_RECOGNIZE     devnet PDA
45 dogs, 27 breeds     Dynamic Tables    epoch to state      syndrome catalogue  portable record
wall-clock replay      1 min lag DAG     dog-disjoint folds  clinical sequences  claim not data
```

Snowflake holds the compute: ten million sensor rows, a declarative transform DAG, supervised classification, row pattern recognition, four ML functions and an LLM, all without leaving SQL.

Solana holds the thing Snowflake cannot provide: portability. A dog arriving at a shelter arrives with no history, because shelters, vets and adopters share no database and never will. The health attestation is published as a signed claim, hashed identifier only, so the record survives rehoming. Publish the claim, never the data.

The five numbers that sell it

Number	What it is
10,611,068	Raw telemetry rows processed, from a published research dataset
45 / 27	Dogs and distinct breeds, each dog its own control
100 Hz	Sample rate, dual sensor: neck collar and back harness
6	Clinical syndromes expressed as row patterns
1 minute	Declared target lag on the transform DAG. No cron anywhere.

2. How to use this document

This document has two audiences and you are both of them.

Section 3 is for the AI. It is a single, self-contained build prompt written to be pasted into Claude Code at the root of an empty repo. It assumes nothing and references the rest of this document only by content, not by file, so it works standalone.

Sections 4 to 23 are for you. They are the spec the AI will be building against, plus the decisions the AI should not be making on its own: which syndromes to encode, what the dashboard looks like, which screenshots you need for the post, and what to cut when Sunday night arrives and something is on fire.

> **Do not paste the whole document into the model.** Paste section 3, then feed sections in as you reach them. A 30-page prompt produces worse code than a one-page prompt followed by focused follow-ups. Build the pipeline first, the dashboard second, the polish third.

The rule that decides every argument with the AI

If a component's only job is to name-drop a technology, delete it. Every stage in this build does load-bearing work. If the AI proposes something whose value is that it "uses Cortex" or "uses Solana," the answer is no.

The three-strikes rule on scope

You have roughly sixty hours and you will lose some of them to a Snowflake error nobody has blogged about. Cut in this order when you fall behind:

1. Shelter Reality tab (section 12) goes first. It is the emotional close, not the substance.
2. Drivers tab and `TOP_INSIGHTS` go second.
3. Solana attestation goes third. You lose one prize category and keep the one you are targeting hardest.

The syndrome layer never gets cut. It is the entire submission. Everything else is context around it.

3. The master build prompt

Paste this into Claude Code in an empty repository. Have the Kaggle dataset downloaded first (section 5) and your Snowflake credentials in a `.env`.

MASTER PROMPT — PASTE AS-IS

```
You are building TELLTAIL, a hackathon submission for the DEV Weekend Challenge
(Dog Days Edition), targeting "Best use of Snowflake" as the primary category and
"Best use of Solana" as secondary. Deadline is hard: Monday 17 Aug 2026, 12:29 IST.

THE PRODUCT
TELLTAIL ingests real dog collar accelerometer telemetry, classifies each second
into a behavioural state, and detects veterinary syndromes as ORDERED SEQUENCES of
states using Snowflake's MATCH_RECOGNIZE row pattern recognition. The thesis: a
threshold cannot detect a syndrome. A vet diagnoses from an ordered sequence of
events, not from a daily average. MATCH_RECOGNIZE makes a differential diagnosis
expressible as a regex over rows. That is the whole submission.

NON-NEGOTIABLE PRINCIPLES
1. Every component does load-bearing work. If a piece exists only to name-drop a
   technology, do not build it. Tell me instead.
2. All heavy compute happens INSIDE Snowflake. No pandas transformation pipelines.
   Python is for loading, replaying, and the Streamlit UI only. If you find
   yourself writing a groupby in pandas, stop: that is a SQL job.
3. Honesty over polish. Anything simplified, seeded, or heuristic gets an explicit
   label in the data (a column, not a comment) and a row in a HONESTY.md table.
4. Small commits, frequently, with real messages. Judging reviews commit history.
5. Cortex AI Function calls are BATCHED into tables by a task and served from cache.
   Never call Cortex from a dashboard render path. Trial accounts are capped at
   roughly ten credits per day of AI Function usage and I will burn it in an hour.

DATA
Kaggle dataset "benjamingray44/inertial-data-for-dog-behaviour-classification",
already downloaded to ./data/. Files: DogMoveData.csv (~10.6M rows, 20 cols,
100 Hz, dual IMU on neck collar and back harness, with post-hoc video-annotated
behaviour labels), DogInfo.csv (45 dogs, 27 breeds, age/weight/height/sex),
Data_description.txt.

STEP ZERO, BEFORE ANY OTHER WORK: read Data_description.txt, then profile
DogMoveData.csv. Print the exact column names, the distinct values in every
behaviour/label column with counts, the per-dog row counts, and the time column
semantics. Do NOT assume column names from my description. Report back what is
actually there and propose the ethogram state vocabulary from the real labels
before writing any DDL. If SCRATCH or SHAKE are not present as labels, say so
clearly, because that changes the syndrome catalogue design.

ARCHITECTURE, TEN STAGES
 1 REPLAY    Python replayer streams the CSV in timestamp order at wall-clock
             speed, 8-second micro-batches, so a static dataset behaves as a live
             feed. Supports --speed to compress time for demos.
 2 LAND      RAW.COLLAR_TELEMETRY, typed columns plus a VARIANT payload column,
             a _batch_id and an is_replay flag.
 3 FEATURES  Dynamic Table, TARGET_LAG '1 minute'. Aggregates 100 Hz into 1-second
             epochs. Per epoch and per sensor: vector magnitude mean/std/min/max,
             signal magnitude area, energy, zero-crossing rate, a jerk proxy via
             LAG, pitch and roll via ATAN2, gyro magnitude. AND CRITICALLY:
             CORR(neck vector magnitude, back vector magnitude) over the epoch.
             That correlation separates neck-dominant behaviours (scratching,
             shaking) from whole-body locomotion in one SQL function, and it is
             only possible because this dataset has two sensors.
 4 STATE     SNOWFLAKE.ML.CLASSIFICATION trained on the labelled epochs, predicting
             the ethogram state. Split train/test BY DOG, never by row: the
             literature reports single-subject classifiers dropping from ~91% to
             ~70-74% when generalising to unseen dogs. Persist the split, the
             confusion matrix and per-class F1 into an ML schema table.
 5 SYNDROME  MATCH_RECOGNIZE over the epoch state sequence. Six clinical patterns
             (I will supply the catalogue). PARTITION BY dog_id ORDER BY epoch_ts,
             ONE ROW PER MATCH, AFTER MATCH SKIP PAST LAST ROW, MEASURES capturing
             MATCH_NUMBER, onset and resolve timestamps and per-symbol counts.
             Each pattern is its own view; UNION ALL into MARTS.SYNDROME_MATCHES.
 6 BASELINE  ASOF JOIN each dog's current epoch features against its own trailing
             baseline, plus a breed-cohort baseline from DogInfo. Every dog is its
             own control. A Husky's normal is a Bulldog's emergency.
 7 ML        SNOWFLAKE.ML.FORECAST and ANOMALY_DETECTION on the per-dog activity
             index, retrained on a task. Strict train/detect boundary split (train
             ts < T, detect ts >= T) or Cortex rejects it. Plus TOP_INSIGHTS
             pointed at the deviation metric with breed, age band, weight band and
             sex as dimensions, to find what actually drives deterioration.
 8 LANGUAGE  AI_COMPLETE writes a SOAP-format veterinary handoff note from the
             syndrome evidence. AI_CLASSIFY assigns triage severity
             (routine / schedule / urgent). AI_AGG writes a pack-wide brief. All
             batched by a task into MARTS tables.
 9 TRUTH     A host-side sync pulls Austin Animal Center intakes and outcomes from
             the Socrata open data API into REF tables, filtered to dogs, so the
             behavioural categories the pipeline detects can be set beside the
             real shelter outcomes those categories produce.
10 ATTEST    ORACLE.PUBLISH_QUEUE staged by a Snowflake task; a Node bridge holds
             the keypair, signs, submits to Solana devnet, writes tx signature and
             explorer URL back. The keypair NEVER touches Snowflake. Payload is a
```

```
             hashed dog id, syndrome code, epoch range, severity and model version.
             Publish the claim, never the data.
11 SERVE     Streamlit in Snowflake, nine tabs. Spec supplied separately.

CRITICAL TECHNICAL CONSTRAINTS, VERIFIED, DO NOT REDISCOVER THESE
- MATCH_RECOGNIZE cannot be used inside a recursive CTE.
- A dynamic table containing MATCH_RECOGNIZE will not refresh incrementally.
  Setting REFRESH_MODE = INCREMENTAL explicitly with an unsupported construct
  fails compilation; AUTO silently resolves to FULL. Drive the syndrome layer with
  a TASK on the epoch layer instead, or accept REFRESH_MODE = FULL and say so.
- NEVER run MATCH_RECOGNIZE against the raw 100 Hz rows. Aggregate to 1-second
  epochs first: 10.6M rows becomes roughly 106K. Pattern matching happens on the
  epoch layer only. This is a hard performance gate.
- Set the account timezone to UTC before anything else:
  ALTER ACCOUNT SET TIMEZONE = 'Etc/UTC';
- Streamlit in Snowflake cannot reach the public internet. Anything external
  (shelter data, price feeds) is written INTO Snowflake by a host-side script and
  the app only ever reads a table.
- Streamlit in Snowflake pins an older Streamlit and pandas than you develop
  against. Guard st.column_config with hasattr. Snowpark to_pandas() returns
  numerics as object-dtype Decimals; convert element-wise with float() in a query
  helper and feed charts plain Python lists, or every line chart renders as an
  identical straight diagonal.
- Declare packages in an environment.yml shipped next to the app file in the stage,
  or plotly will ModuleNotFoundError in SiS only.
- External access integrations are not supported on Snowflake trial accounts. The
  Solana bridge must use the outbound queue-poll design, not an inbound proc call.

BUILD ORDER, AND STOP AT EACH GATE FOR MY REVIEW
GATE A: dataset profile + proposed ethogram vocabulary. No DDL yet.
GATE B: schemas, RAW landing, replayer working, rows visibly arriving.
GATE C: epoch feature Dynamic Table refreshing, with the neck/back correlation
        column populated and sane. Show me a sample.
GATE D: classification trained, confusion matrix printed, states populated.
GATE E: ONE syndrome pattern matching on real data, with the matched rows shown.
        This is the riskiest thing in the build. Prove it early.
GATE F: full syndrome catalogue + ML functions.
GATE G: Cortex layer batched into tables.
GATE H: Streamlit app, tab by tab.
GATE I: Solana bridge, Austin sync, README, HONESTY.md.

DELIVERABLES
- warehouse/*.sql, numbered, idempotent, runnable start to finish by one script
- ingest/replay.py, scripts/load_raw.py, scripts/austin_sync.py
- warehouse/streamlit_app.py + environment.yml
- bridge/ (Node, Solana devnet publisher)
- scripts/run_sql.py --all, scripts/demo_spike.py, scripts/deploy_streamlit.py
- README.md with a five-command quickstart, HONESTY.md with the simplifications
- .env.example

Start with GATE A now. Profile the data and report back. Do not write DDL yet.
```

Follow-up prompts to keep in your back pocket

When	Prompt
After Gate C	"Show me ten epochs where neck/back correlation is below 0.2 and ten where it is above 0.8, with the true labels. If the correlation does not separate them, the feature is wrong and I want to know now."
At Gate E	"Before tuning, run the pattern with the quantifiers relaxed by one and tell me how many matches each variant produces per dog. I want to see the sensitivity curve, not a single hand-tuned number."
Any Cortex work	"How many AI Function calls will this task make per run, and what does that cost in credits per hour? I am on a trial cap."
Dashboard	"Build tab N only. Do not touch other tabs. Show me the query it runs before you write the chart."
When stuck	"Stop. Write down what you expected, what happened, and the smallest query that reproduces it. Then fix that query alone."

4. Environment and prerequisites

Accounts you need, in order of lead time

Thing	Notes
Snowflake trial	30 days, 400 credits. Choose a region where Cortex AI Functions are natively available, or enable cross-region inference. AWS US West (Oregon) is the safest default. Trial accounts without a payment method are capped at roughly ten credits per day of AI Function usage, which is the single most likely thing to derail Sunday.
Kaggle account	Needed for the dataset and the API token. Two minutes.
Solana devnet keypair	Generate locally, airdrop devnet SOL. No account needed.
Socrata app token	Optional. Austin's open data API works without one at lower rate limits, which is fine for a one-shot sync.

Account bootstrap, run this first

```
-- 00_account_setup.sql  (run as ACCOUNTADMIN)
ALTER ACCOUNT SET TIMEZONE = 'Etc/UTC';           -- block times are UTC; do not skip
ALTER ACCOUNT SET CORTEX_ENABLED_CROSS_REGION = 'ANY_REGION';

CREATE WAREHOUSE IF NOT EXISTS TELLTAIL_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE DATABASE IF NOT EXISTS TELLTAIL;

GRANT DATABASE ROLE SNOWFLAKE.CORTEX_USER TO ROLE SYSADMIN;

-- sanity checks: run these before you build anything on top
SELECT SNOWFLAKE.CORTEX.COMPLETE('claude-3-5-sonnet', 'Say OK');
SELECT CURRENT_REGION(), CURRENT_ACCOUNT(), CURRENT_TIMESTAMP();
```

> **Ninety-second smoke test, do this before writing a line of code.** Run a trivial `MATCH_RECOGNIZE` against a five-row inline table, and a trivial `SNOWFLAKE.CORTEX.COMPLETE`. If either fails on your account or region, you need to know at hour zero, not hour thirty. The `MATCH_RECOGNIZE` smoke test is in section 9.

Local toolchain

Tool	Version	Why
Python	3.11	Loader, replayer, sync scripts, SiS deploy
snowflake-connector-python	latest, with `[pandas]` extra	Bulk load and SQL execution
snowflake-snowpark-python	latest	Streamlit in Snowflake session
snowflake-cli	latest	Easiest path to deploy the Streamlit app to a stage
pandas, pyarrow	latest	Chunked CSV read and Parquet conversion
Node	20 LTS	Solana bridge (reuse your FERVOR bridge wholesale)
@solana/web3.js	1.x	Devnet transaction signing
kaggle CLI	latest	Dataset download

```
python -m venv .venv && source .venv/bin/activate
pip install "snowflake-connector-python[pandas]" snowflake-snowpark-python \
            snowflake-cli pandas pyarrow python-dotenv requests kaggle
npm install @solana/web3.js dotenv
```

.env.example

```
SNOWFLAKE_ACCOUNT=xy12345.us-west-2
SNOWFLAKE_USER=
SNOWFLAKE_PASSWORD=
SNOWFLAKE_ROLE=SYSADMIN
SNOWFLAKE_WAREHOUSE=TELLTAIL_WH
SNOWFLAKE_DATABASE=TELLTAIL

SOLANA_RPC_URL=https://api.devnet.solana.com
```

```
SOLANA_KEYPAIR_PATH=./secrets/devnet.json
TELLTAIL_PROGRAM_ID=            # leave blank; bridge writes Memo txs until set

SOCRATA_APP_TOKEN=              # optional
REPLAY_SPEED=1                  # 1 = wall clock, 60 = one minute per second
```

environment.yml for Streamlit in Snowflake

This file ships to the stage next to the app. Without it, plotly fails to import in SiS only, which is a confusing hour of your life.

```
name: sf_env
channels:
  - snowflake
dependencies:
  - python=3.11
  - plotly
  - pandas
  - numpy
```

5. The dataset

Provenance

Kaggle: `benjamingray44/inertial-data-for-dog-behaviour-classification`, mirrored from Mendeley Data. Originally published as Vehkaoja et al., "Description of Movement Sensor Dataset for Dog Behavior Classification," *Data in Brief*, 2022, University of Helsinki.

Property	Value
Rows	10,611,068
Columns	20
Subjects	45 middle-to-large dogs, 27 breeds, mean age 4.9 years, mean weight 24.5 kg
Sensors	Two ActiGraph GT9X Link units: one on the neck collar, one on the back harness
Modalities	3D accelerometer + 3D gyroscope per sensor (six degrees of freedom each)
Sample rate	100 Hz per channel
Labels	Post-hoc video annotation at one-second resolution, synchronised to the raw signal
Core tasks	Sitting, standing, lying on chest, trotting, walking, playing, treat-searching (sniffing), galloping
Repeats	17 of the 45 dogs did the full protocol twice, giving test-retest data
Files	`DogMoveData.csv`, `DogInfo.csv`, `Data_description.txt`

```
kaggle datasets download -d benjamingray44/inertial-data-for-dog-behaviour-classification
unzip inertial-data-for-dog-behaviour-classification.zip -d ./data/
```

> **Read `Data_description.txt` before writing DDL, and make the AI do the same.** Column naming in this dataset follows a neck/back and accel/gyro convention, and there are multiple behaviour annotation columns rather than one. Profile the distinct values with counts and build the ethogram vocabulary from what is genuinely in the file. Guessing column names here costs you an hour of silent nulls.

The scratch problem, and the honest answer

The core protocol in this dataset covers locomotion and posture. Fine-grained health behaviours like scratching and head shaking may not appear as first-class labels. This matters because two of the six syndromes depend on them.

Handle it like this, in order of preference:

1. **If the labels exist** in a secondary behaviour column, use them and say nothing more about it.
2. **If they do not**, derive them with an explicitly labelled heuristic: a neck-dominant, high-frequency, low-correlation epoch is a shake or scratch candidate. The neck/back correlation feature was designed for exactly this. Write the derived state into a column named so nobody can miss it, for example `state_source = 'HEURISTIC'` versus `'MODEL'`, and put a row in the honesty table.
3. **Do not** quietly relabel and hope. Judges reward the labelled compromise and punish the hidden one.

Loading strategy

A 10.6 million row CSV is a fine Snowflake workload and a poor pandas workload. Load it once, properly, and never touch the raw file again.

```
# scripts/load_raw.py, sketch
# 1. read the CSV in chunks, write Parquet shards (roughly 500K rows each)
# 2. PUT the shards to an internal stage with AUTO_COMPRESS
# 3. COPY INTO RAW.COLLAR_TELEMETRY_BULK from the stage
# 4. verify: SELECT COUNT(*), COUNT(DISTINCT dog_id), MIN(t_sec), MAX(t_sec)

PUT file://./data/parquet/*.parquet @TELLTAIL.RAW.DOG_STAGE AUTO_COMPRESS=TRUE;

COPY INTO TELLTAIL.RAW.COLLAR_TELEMETRY_BULK
FROM @TELLTAIL.RAW.DOG_STAGE
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
ON_ERROR = ABORT_STATEMENT;
```

> **Bulk table and live table are two different things, deliberately.** `COLLAR_TELEMETRY_BULK` is the full historical corpus, loaded once, used for model training and for the deep-history tabs. `COLLAR_TELEMETRY` is the live landing table the replayer feeds in 8-second micro-batches. The DAG watches the live table. This is what makes the demo feel alive instead of static, and it mirrors exactly what your Helius poller did last time.

6. Warehouse design

One database, seven schemas, data flowing strictly left to right. Every schema has a single job and nothing reaches backwards.

Schema	Purpose	Key objects
`REF`	Hand-loaded seed and external truth	`DOG_INFO`, `ETHOGRAM`, `SYNDROME_CATALOGUE`, `BREED_COHORT`, `PARAMS`, `AAC_INTAKES`, `AAC_OUTCOMES`
`RAW`	Untouched landing	`COLLAR_TELEMETRY` (live), `COLLAR_TELEMETRY_BULK` (historical)
`STAGING`	Epoch features (Dynamic Tables)	`EPOCH_FEATURES`, `EPOCH_LABELS`
`MARTS`	States, syndromes, baselines	`EPOCH_STATES`, `SYNDROME_MATCHES`, `DOG_BASELINE`, `DOG_DEVIATION`, `STATE_TRANSITIONS`, `PACK_STATUS`
`ML`	Model artefacts and outputs	`STATE_MODEL`, `MODEL_EVAL`, `ACTIVITY_HISTORY`, `ACTIVITY_FORECAST`, `ACTIVITY_ANOMALIES`, `DRIVER_INSIGHTS`
`AI`	Cached Cortex output	`VET_NOTES`, `TRIAGE`, `PACK_BRIEF`
`ORACLE`	Attestation queue and audit	`PUBLISH_QUEUE`, `PUBLISH_LOG`

Lineage

```
RAW.COLLAR_TELEMETRY  (100 Hz, live micro-batches)
   |
   +-- STAGING.EPOCH_FEATURES        [Dynamic Table, TARGET_LAG '1 minute']
          |                           1-second epochs, dual-sensor features
          |
          +-- MARTS.EPOCH_STATES     [Dynamic Table]  ML.CLASSIFICATION applied
                 |
                 +-- MARTS.SYNDROME_MATCHES   [TASK, 2 min]  MATCH_RECOGNIZE
                 +-- MARTS.STATE_TRANSITIONS  [Dynamic Table] behavioural Markov
                 +-- MARTS.DOG_BASELINE       [Dynamic Table] ASOF JOIN self-compare
                        |
                        +-- ML.ACTIVITY_HISTORY   [TASK, 1 min snapshot]
                        +-- ML.ACTIVITY_FORECAST  [TASK, 5 min]
                        +-- ML.ACTIVITY_ANOMALIES [TASK, 5 min]
                        +-- ML.DRIVER_INSIGHTS    [TASK, 15 min] TOP_INSIGHTS
                               |
                               +-- AI.VET_NOTES    [TASK, 5 min] AI_COMPLETE
                               +-- AI.TRIAGE       [TASK, 5 min] AI_CLASSIFY
                               +-- AI.PACK_BRIEF   [TASK, 15 min] AI_AGG
                                      |
                                      +-- ORACLE.PUBLISH_QUEUE [TASK, 5 min]
                                             |
                                             +-- Node bridge --> Solana devnet
```

> Screenshot the Snowsight lineage view of this DAG once it is live. It was one of the strongest images in the FERVOR post because it proves the architecture diagram is not aspirational.

7. Stages 1 to 3: replay, land, features

Stage 1: the replayer

The replayer is what turns a Kaggle download into a demo. It reads the bulk table in timestamp order and inserts into the live landing table at wall-clock speed, in 8-second micro-batches, exactly as your Helius poller did.

```
# ingest/replay.py, core loop
# --speed 1   real time (one second of dog time per second)
# --speed 60  one minute of dog time per second, for recording the demo
# --dogs 12   restrict to a subset so the pack view stays readable

while True:
    window = fetch_next_window(cursor_ts, seconds=8 * speed)
    if not window: break
    insert_batch(window, batch_id=f"REPLAY_{run_id}", is_replay=True)
    cursor_ts = window.max_ts
    sleep(8)
```

Two flags earn their place. `--speed` lets you record a four-minute demo video of four hours of dog behaviour. `--dogs` keeps the pack view legible in screenshots: forty-five cards is a wall, twelve is a dashboard.

Stage 2: the landing table

```
CREATE OR REPLACE TABLE RAW.COLLAR_TELEMETRY (
  dog_id        NUMBER,
  test_num      NUMBER,
  t_sec         FLOAT,             -- seconds from session start
  sample_ts     TIMESTAMP_NTZ,     -- t_sec projected onto a wall-clock replay epoch
  neck_ax FLOAT, neck_ay FLOAT, neck_az FLOAT,
  neck_gx FLOAT, neck_gy FLOAT, neck_gz FLOAT,
  back_ax FLOAT, back_ay FLOAT, back_az FLOAT,
  back_gx FLOAT, back_gy FLOAT, back_gz FLOAT,
  label_primary   STRING,          -- exact names TBD after profiling
  label_secondary STRING,
  label_tertiary  STRING,
  raw_payload   VARIANT,           -- full row as JSON, untouched
  _batch_id     STRING,
  is_replay     BOOLEAN DEFAULT TRUE,
  landed_at     TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Stage 3: the epoch feature layer

This is where 10.6 million rows become roughly 106 thousand, and where the one feature that makes this project technically distinctive gets computed.

```
CREATE OR REPLACE DYNAMIC TABLE STAGING.EPOCH_FEATURES
  TARGET_LAG = '1 minute'
  WAREHOUSE  = TELLTAIL_WH
AS
WITH s AS (
  SELECT
    dog_id, test_num,
    TIME_SLICE(sample_ts, 1, 'SECOND')                    AS epoch_ts,
    SQRT(neck_ax*neck_ax + neck_ay*neck_ay + neck_az*neck_az) AS vm_neck,
    SQRT(back_ax*back_ax + back_ay*back_ay + back_az*back_az) AS vm_back,
    SQRT(neck_gx*neck_gx + neck_gy*neck_gy + neck_gz*neck_gz) AS gm_neck,
    ABS(neck_ax) + ABS(neck_ay) + ABS(neck_az)            AS sma_neck,
    ATAN2(neck_ax, SQRT(neck_ay*neck_ay + neck_az*neck_az)) AS pitch_neck,
    ATAN2(neck_ay, SQRT(neck_ax*neck_ax + neck_az*neck_az)) AS roll_neck,
    label_primary, label_secondary
  FROM RAW.COLLAR_TELEMETRY
)
SELECT
  dog_id, test_num, epoch_ts,
  COUNT(*)                          AS n_samples,      -- expect ~100
  AVG(vm_neck)                      AS vm_neck_mean,
  STDDEV(vm_neck)                   AS vm_neck_std,
  MAX(vm_neck) - MIN(vm_neck)       AS vm_neck_range,
  SUM(vm_neck * vm_neck)            AS energy_neck,
  AVG(sma_neck)                     AS sma_neck,
  AVG(vm_back)                      AS vm_back_mean,
  STDDEV(vm_back)                   AS vm_back_std,
  AVG(gm_neck)                      AS gyro_neck_mean,
  STDDEV(pitch_neck)                AS pitch_var,
  STDDEV(roll_neck)                 AS roll_var,

  -- THE FEATURE. Neck-dominant behaviours decouple the two sensors.
  -- Scratching and head shaking: correlation near zero.
  -- Trotting and walking: both sensors move in phase, correlation high.
  CORR(vm_neck, vm_back)            AS neck_back_corr,

  MODE(label_primary)               AS label_primary,
  MODE(label_secondary)             AS label_secondary
```

```
FROM s
GROUP BY dog_id, test_num, epoch_ts;
```

> **Why the correlation column is the technical star.** Almost every published approach to this dataset feeds a big feature vector into a black-box classifier. One `CORR()` across two sensor streams gives you a physically interpretable discriminator that separates the exact behaviours the health syndromes depend on, and it exists only because this dataset has two sensors and you read the description. When you write the post, this is the paragraph where a Snowflake judge decides you actually know what you are doing.

Validate the feature before you trust it

```
SELECT label_primary,
       COUNT(*)                      AS epochs,
       ROUND(AVG(neck_back_corr), 3)  AS avg_corr,
       ROUND(STDDEV(neck_back_corr), 3) AS sd_corr
FROM STAGING.EPOCH_FEATURES
WHERE label_primary IS NOT NULL
GROUP BY 1 ORDER BY 3;
```

If locomotion labels do not sit meaningfully above posture and neck-dominant labels, the feature is not doing what you think and you need to know at Gate C, not in the write-up.

8. Stage 4: the ethogram classifier

Every epoch gets exactly one state. The state vocabulary comes from the real labels found at Gate A, mapped into a clinical ethogram.

State	Source	Used by syndrome
`REST`	Lying on chest	Ear irritation, exercise intolerance, reluctance to rise
`SIT`	Sitting	Reluctance to rise
`STAND`	Standing	Reluctance to rise, GI discomfort
`WALK`	Walking	Intermittent lameness
`TROT`	Trotting	Exercise intolerance
`GALLOP`	Galloping	Exercise intolerance
`SNIFF`	Treat-searching	GI discomfort
`PLAY`	Playing	Baseline context
`SHAKE`	Label if present, else heuristic (flagged)	Ear irritation
`SCRATCH`	Label if present, else heuristic (flagged)	Ear irritation
`PAUSE`	Derived: locomotion interrupted by a still epoch	Intermittent lameness

The split that separates you from the field

> **Split by dog, never by row.** The published literature on this dataset reports single-subject classifiers falling from around 91% accuracy to roughly 70–74% when generalising to dogs they were not trained on. If you split randomly by row, adjacent samples from the same dog land on both sides and your reported accuracy is a fiction. Hold out entire dogs. Report the honest number. Then say in the post exactly why you did it, and you have instantly out-rigoured every other entry in the pool.

```
-- 35 dogs train, 10 dogs held out entirely
CREATE OR REPLACE VIEW ML.V_TRAIN AS
SELECT * FROM STAGING.EPOCH_FEATURES
WHERE label_primary IS NOT NULL
  AND dog_id NOT IN (SELECT dog_id FROM REF.HOLDOUT_DOGS);

CREATE OR REPLACE SNOWFLAKE.ML.CLASSIFICATION ML.STATE_MODEL(
  INPUT_DATA       => SYSTEM$QUERY_REFERENCE('SELECT * FROM TELLTAIL.ML.V_TRAIN'),
  TARGET_COLNAME   => 'LABEL_PRIMARY'
);

-- apply, and persist the evaluation so the dashboard can show it
CREATE OR REPLACE TABLE ML.MODEL_EVAL AS
SELECT * FROM TABLE(ML.STATE_MODEL!SHOW_EVALUATION_METRICS());

CREATE OR REPLACE TABLE ML.FEATURE_IMPORTANCE AS
SELECT * FROM TABLE(ML.STATE_MODEL!SHOW_FEATURE_IMPORTANCE());
```

`SHOW_FEATURE_IMPORTANCE` is a free win: if `neck_back_corr` ranks high, you have a chart that proves the feature you invented is the one the model leans on. That is a screenshot.

Fallback if CLASSIFICATION is unavailable on your account

Do not lose the day. Ship a transparent SQL rules ethogram over the same features, label it honestly as a rules classifier, and keep moving. The syndromes are the submission, not the classifier.

```
CASE
  WHEN vm_neck_std < 0.05 AND pitch_var < 0.02                    THEN 'REST'
  WHEN vm_neck_std > 0.9  AND neck_back_corr < 0.25               THEN 'SHAKE'
  WHEN vm_neck_std BETWEEN 0.4 AND 0.9 AND neck_back_corr < 0.35  THEN 'SCRATCH'
  WHEN neck_back_corr > 0.7 AND vm_neck_mean > 1.6                THEN 'GALLOP'
  WHEN neck_back_corr > 0.6 AND vm_neck_mean > 1.25               THEN 'TROT'
  WHEN neck_back_corr > 0.5 AND vm_neck_mean > 1.08               THEN 'WALK'
  ELSE 'STAND'
END AS state
```

9. Stage 5: the syndrome catalogue

This is the submission. Everything before it is plumbing and everything after it is presentation. Spend your best hours here.

The smoke test, run it at hour zero

```
-- Does MATCH_RECOGNIZE work on this account at all? Ninety seconds.
WITH t AS (
  SELECT * FROM VALUES
    (1,1,'REST'),(1,2,'SHAKE'),(1,3,'SCRATCH'),(1,4,'SCRATCH'),
    (1,5,'SCRATCH'),(1,6,'SHAKE'),(1,7,'SCRATCH'),(1,8,'SCRATCH')
  AS v(dog_id, i, state)
)
SELECT * FROM t
MATCH_RECOGNIZE (
  PARTITION BY dog_id ORDER BY i
  MEASURES MATCH_NUMBER() AS m, FIRST(onset.i) AS start_i, LAST(itch.i) AS end_i,
           COUNT(itch.*) AS scratch_epochs
  ONE ROW PER MATCH
  AFTER MATCH SKIP PAST LAST ROW
  PATTERN ( onset shake itch{3,} shake itch{2,} )
  DEFINE onset AS state = 'REST', shake AS state = 'SHAKE', itch AS state = 'SCRATCH'
);
-- expect exactly one row: m=1, start_i=1, end_i=8, scratch_epochs=5
```

The six syndromes

S1 — Otitis / ear irritation

Pattern: `REST SHAKE SCRATCH{3,} SHAKE SCRATCH{2,}`

Why a sequence and not a threshold: daily scratch count is normal in a flea-free dog. The clinical signal is the *alternation* of head shake and scratch bout, emerging from rest. A totals-based tracker cannot see this at all.

S2 — Intermittent lameness

Pattern: `WALK{3,} PAUSE WALK{1,3} PAUSE WALK{1,3} PAUSE`

Why: step count is unchanged. Stride *interruption frequency* is rising. This is the presentation an owner describes as "he seems fine, he just stops a lot now."

S3 — Exercise intolerance

Pattern: `TROT+ REST{5,} TROT{1,2} REST{8,}`

Why: same total activity minutes, collapsed into shorter bursts with progressively longer recoveries. Cardiac and respiratory presentations look exactly like this and are invisible to a daily total.

S4 — Reluctance to rise (osteoarthritis onset)

Pattern: `REST{10,} SLOW_TRANSITION STAND REST{10,}`

Why: the totals are unchanged; the *transition* is slow. Define `SLOW_TRANSITION` as an epoch where pitch variance rises but vector magnitude stays low, which is a dog levering itself up rather than standing.

S5 — Separation distress

Pattern: `STAND PACE{4,} STAND PACE{4,}`

Why: pacing exists in happy dogs. Cyclic pacing punctuated by alert stands, repeating, does not. This is the syndrome that connects directly to section 12, because behaviour is a named surrender reason in real shelter data.

S6 — GI discomfort

Pattern: `SNIFF{5,} CIRCLE{2,} SNIFF{5,}`

Why: repeated pre-elimination posture without resolution. Define `CIRCLE` from sustained yaw rotation in the gyroscope with low forward translation.

Reference implementation

```
CREATE OR REPLACE VIEW MARTS.V_SYNDROME_S1 AS
SELECT 'S1' AS syndrome_code, 'Otitis / ear irritation' AS syndrome_name, m.*
FROM (
  SELECT dog_id, epoch_ts, state FROM MARTS.EPOCH_STATES
) MATCH_RECOGNIZE (
  PARTITION BY dog_id
  ORDER BY epoch_ts
  MEASURES
    MATCH_NUMBER()              AS match_id,
    FIRST(onset.epoch_ts)       AS onset_ts,
```

```
    LAST(itch.epoch_ts)         AS resolve_ts,
    COUNT(itch.*)               AS scratch_epochs,
    COUNT(shake.*)              AS shake_epochs,
    DATEDIFF('second', FIRST(onset.epoch_ts), LAST(itch.epoch_ts)) AS duration_s
  ONE ROW PER MATCH
  AFTER MATCH SKIP PAST LAST ROW
  PATTERN ( onset shake itch{3,} shake itch{2,} )
  DEFINE
    onset AS state = 'REST',
    shake AS state = 'SHAKE',
    itch  AS state = 'SCRATCH'
) m;

-- ... S2 through S6 follow the same shape ...

CREATE OR REPLACE TABLE MARTS.SYNDROME_MATCHES AS
SELECT * FROM MARTS.V_SYNDROME_S1
UNION ALL SELECT * FROM MARTS.V_SYNDROME_S2
UNION ALL SELECT * FROM MARTS.V_SYNDROME_S3
UNION ALL SELECT * FROM MARTS.V_SYNDROME_S4
UNION ALL SELECT * FROM MARTS.V_SYNDROME_S5
UNION ALL SELECT * FROM MARTS.V_SYNDROME_S6;

-- refreshed by a task, NOT a dynamic table (see constraint below)
CREATE OR REPLACE TASK MARTS.T_SYNDROMES
  WAREHOUSE = TELLTAIL_WH SCHEDULE = '2 minute'
AS CREATE OR REPLACE TABLE MARTS.SYNDROME_MATCHES AS ( ... );
```

Three constraints that will cost you an hour each if you rediscover them.
1. `MATCH_RECOGNIZE` cannot appear inside a recursive CTE.
2. A dynamic table containing it will not refresh incrementally. Explicit `REFRESH_MODE = INCREMENTAL` fails compilation with an unsupported construct; `AUTO` silently resolves to `FULL`. Use a task for this layer, or accept `FULL` and document it.
3. Never point it at the 100 Hz raw table. Epoch layer only. 10.6M rows becomes ~106K and the query returns in seconds instead of timing out.

Sensitivity, which is the difference between a demo and a result

A single hand-tuned quantifier is a magic number and judges can smell it. Run each pattern at three quantifier settings and publish the curve.

```
-- how many matches per dog at each strictness level?
SELECT variant, dog_id, COUNT(*) AS matches
FROM MARTS.SYNDROME_SENSITIVITY
GROUP BY 1,2 ORDER BY 1,3 DESC;
-- variant: 'loose' {2,}  'tuned' {3,}  'strict' {5,}
```

That table becomes a chart on the Syndromes tab and a paragraph in the post that says: here is what happens when you relax the pattern, here is where it starts firing on noise, and here is why the tuned setting sits where it does. That paragraph is worth more than three extra features.

10. Stages 6 to 7: baselines and ML functions

Every dog is its own control

A Husky doing 40 minutes of galloping is a Tuesday. A twelve-year-old Bulldog doing the same is an emergency. Population averages are useless here, which is why the baseline layer exists.

```
-- ASOF JOIN each epoch to the same dog's trailing baseline window
SELECT
  cur.dog_id, cur.epoch_ts, cur.activity_index,
  base.activity_index                                  AS baseline_index,
  (cur.activity_index - base.activity_index)
    / NULLIF(base.activity_std, 0)                     AS z_self,
  (cur.activity_index - coh.cohort_mean)
    / NULLIF(coh.cohort_std, 0)                        AS z_cohort
FROM MARTS.ACTIVITY_EPOCH cur
ASOF JOIN MARTS.ACTIVITY_BASELINE base
  MATCH_CONDITION (cur.epoch_ts >= base.window_end)
  ON cur.dog_id = base.dog_id
LEFT JOIN REF.BREED_COHORT coh ON coh.cohort_id = cur.cohort_id;
```

`ASOF JOIN` earns its place for the same reason it did in FERVOR: a naive `LAG(n)` counts rows, not time, so one gap in the feed silently turns your window into a different window.

The three ML functions

Function	Input	What it produces in the UI
`FORECAST`	Per-dog activity index time series	A dumbbell chart: current index to projected index, so you read who is declining at a glance
`ANOMALY_DETECTION`	Same series, strict train/detect split	Flagged points on the timeline, with the demo spike script to make it fire on camera
`TOP_INSIGHTS`	Deviation metric, dimensions: breed, age band, weight band, sex, sensor	The Drivers tab. Produces a finding rather than a chart.

> **The anomaly detection gotcha you already paid for once.** Training and detection windows must not overlap. Split both at the same boundary, strict `<` on the training side and `>=` on the detection side, or Cortex rejects it with a message about evaluation timestamps needing to follow the fitting data. Reuse your FERVOR pattern verbatim.

The demo spike, with the same honesty mechanism as last time

`scripts/demo_spike.py` injects a labelled deterioration into a chosen dog upstream in the raw table, so the entire real DAG computes the consequence. The injected rows carry `is_synthetic = TRUE`, detection sees them, training never fits them, and `--clean` removes them. This is what lets the anomaly detector fire during your demo video without you lying about it.

11. Stage 8: the Cortex AISQL layer

FERVOR's Cortex prompt asked the model to hype up a fanbase. That was correct for that project and would be badly wrong for this one. The tonal register here is clinical, and the tone shift is itself a signal of care.

The vet handoff note

```
CREATE OR REPLACE TABLE AI.VET_NOTES AS
SELECT
  s.dog_id, s.syndrome_code, s.onset_ts,
  AI_COMPLETE('claude-3-5-sonnet',
    'You are writing a veterinary handoff note in SOAP format from wearable ' ||
    'sensor evidence. Be clinical and concise. Do not diagnose; describe ' ||
    'findings and recommend. Four short sections: Subjective, Objective, ' ||
    'Assessment, Plan.' ||
    '\n\nDog: ' || d.breed || ', ' || d.age_years || 'y, ' || d.weight_kg || 'kg.' ||
    '\nPattern detected: ' || c.syndrome_name ||
    '\nSequence: ' || c.pattern_text ||
    '\nOnset: ' || s.onset_ts || ', duration ' || s.duration_s || 's.' ||
    '\nEvidence: ' || s.evidence_json ||
    '\nDeviation from own 7-day baseline: ' || b.z_self || ' SD.' ||
    '\nBreed cohort deviation: ' || b.z_cohort || ' SD.'
  ) AS soap_note
FROM MARTS.SYNDROME_MATCHES s
JOIN REF.DOG_INFO d           ON d.dog_id = s.dog_id
JOIN REF.SYNDROME_CATALOGUE c ON c.syndrome_code = s.syndrome_code
JOIN MARTS.DOG_DEVIATION b    ON b.dog_id = s.dog_id;
```

Triage classification

```
AI_CLASSIFY(
  soap_note,
  ['routine monitoring', 'schedule appointment', 'urgent veterinary attention'],
  {'task_description': 'Triage urgency of a canine wearable-sensor finding'}
) AS triage
```

Pack-wide brief

```
AI_AGG(soap_note,
  'Across this pack, name the dog most in need of attention and why, ' ||
  'the most common emerging pattern, and one thing a caretaker should ' ||
  'check today. Three sentences. No preamble.') AS pack_brief
```

> **Budget discipline, non-negotiable.** Trial accounts without a payment method are capped at roughly ten credits per day of Cortex AI Function usage. Batch every call into a table on a task, dedupe on `(dog_id, syndrome_code, onset_ts)` so a note is generated once and never regenerated, and have the dashboard read the cached table. If you wire a "generate" button, gate it behind an explicit click and cache the result.

Optional stretch, only if you are ahead on Sunday morning

Snowflake's multimodal AI Functions handle images, documents and audio from a stage. Two extensions are genuinely on-theme:

- **Bark audio.** `AI_CLASSIFY` over staged bark clips to add a vocalisation channel to the ethogram. Public bark corpora exist with context labels (stranger, play, isolation, distress). This makes the pipeline multimodal and is a two-line addition if the plumbing already works.
- **Dog photos.** `AI_COMPLETE` multimodal to generate the roster card description from an image. Cosmetic, but it makes the Pack tab look finished.

Both are strictly bonus. If the syndrome layer is not perfect, neither of these gets built.

12. Stage 9: ground truth from real shelter data

In FERVOR the World Cup tab answered the only objection that mattered: is any of this data actually real. Here the equivalent is stronger, because it answers a different question: does any of this actually matter.

Austin Animal Center publishes intakes and outcomes through the City of Austin open data portal, covering October 2013 onward, hundreds of thousands of records, updated continuously. **Behaviour is a named outcome reason in that data, sitting alongside aggression and medical.** Behavioural problems are among the reasons dogs are surrendered and, at the far end, euthanised.

```
# scripts/austin_sync.py, two endpoints, two REF tables
INTAKES  = "https://data.austintexas.gov/resource/wter-evkm.json"
OUTCOMES = "https://data.austintexas.gov/resource/9t4d-g238.json"
params   = {"$limit": 50000, "$where": "animal_type='Dog'"}
# SiS cannot reach the internet, so the host writes and the app only reads,
# exactly like football_sync.py did last time.
```

What the tab actually shows

1. Intake reasons for dogs over time, with behaviour-linked categories highlighted.
2. Median length of stay by intake condition and breed group, so the cost of a behavioural label is visible in days.
3. Outcome distribution, with the honest framing that Austin is a no-kill shelter and over ninety percent of animals are adopted, transferred or returned, which makes the behavioural tail all the more pointed.
4. The punchline chart: your six syndrome categories on one axis, the shelter's behaviour-linked outcome counts on the other. Your pipeline detects, on a collar, at home, the same categories that are recorded at intake after the relationship has already broken down.

> **The closing line of the post writes itself here.** The dog was showing the pattern for eleven days before anyone noticed. The warehouse noticed on day two. That is the difference between a vet visit and a surrender form.

13. Stage 10: the Solana attestation bridge

One tab, one argument, and reuse of code you have already written and debugged.

The argument, in the form a judge can check

A dog arriving at a shelter arrives with no history. Vets, shelters, boarding kennels and adopters share no database and commercially never will. A health record that lives in one vendor's cloud dies when the dog changes hands, which is precisely the moment the record matters most. So the attestation is published as a signed claim on a public chain: hashed identifier, syndrome code, epoch range, severity, model version. **Publish the claim, never the data.**

```
CREATE OR REPLACE TABLE ORACLE.PUBLISH_QUEUE (
  publish_id     NUMBER IDENTITY,
  dog_hash       STRING,          -- SHA2(dog_id || salt), never the raw id
  syndrome_code  STRING,
  severity       NUMBER,          -- 1 routine, 2 schedule, 3 urgent
  onset_ts       TIMESTAMP_NTZ,
  duration_s     NUMBER,
  model_version  STRING,
  status         STRING DEFAULT 'PENDING',  -- PENDING -> SENT -> CONFIRMED | FAILED
  queued_at      TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  tx_signature   STRING,
  explorer_url   STRING,
  last_error     STRING
);
```

```
{
  "oracle":  "TELLTAIL",
  "source":  "Snowflake MARTS.SYNDROME_MATCHES",
  "subject": "a7f3c1...",
  "finding": "S1",
  "severity": 2,
  "window":  "2026-08-16T14:02:11Z/PT47S",
  "model":   "state-v3+match-v1"
}
```

The security property to state plainly, because it is the same one that impressed last time: **the keypair never touches Snowflake.** Snowflake stages numbers in a queue table; a small Node bridge holds the key, signs, submits, and writes the audit trail back. Keep the outbound queue-poll design, because external access integrations are not supported on trial accounts.

Memo transactions first. If the Anchor program deploys with time to spare, flip to a PDA per subject. If not, memo transactions are real, confirmed and explorer-visible, and you say exactly that.

14. The dashboard: nine tabs, every chart

Streamlit in Snowflake. No external hosting, the app runs where the data lives, which is itself part of the pitch.

Tab 1 — The Pack

The ward round. First thing a judge sees.

- Card grid, one per dog: breed, age, weight, photo or breed glyph, current state chip, triage badge (green / amber / red), and a 60-epoch activity sparkline.
- Sort control: by triage severity, by deviation from own baseline, by name.
- Top strip of four metrics: dogs monitored, epochs processed today, open syndrome findings, mean detection latency.
- Right rail: the Cortex pack brief, three sentences, cached.

Tab 2 — Live Collar

Proof the thing is actually running. This is the tab that kills the "is it real" objection.

- Dog selector, then three stacked charts sharing an x-axis:
  1. Raw 100 Hz waveform, neck and back overlaid in two colours, last 30 seconds.
  2. Derived epoch features: vector magnitude mean, standard deviation, and neck/back correlation as a separate line pinned to a -1 to 1 axis.
  3. Classified state ribbon: a horizontal band of coloured blocks, one per second.
- Auto-refresh every 15 seconds while the replayer runs.
- Caption showing rows landed in the last minute, so the live-ness is a number and not a vibe.

Tab 3 — Ethogram

The behavioural fingerprint of an individual dog.

- 24-hour state ribbon, full width, one colour per state.
- State-transition matrix as a heatmap: which behaviour follows which. This is a Markov chain computed in SQL and it looks like real science because it is.
- State budget donut: proportion of the day in each state, with the dog's own trailing average ghosted behind it.
- Bout-length distribution histogram per state, which is where lameness and exercise intolerance become visible before any syndrome fires.

Tab 4 — Syndromes

The money tab. Build this one twice as carefully as the others.

- Table of every match: dog, syndrome, onset, duration, symbol counts, confidence.
- Click a row and the epoch timeline below highlights **the exact matched rows**, colour-coded by pattern symbol, with the pattern string printed above it. Seeing `onset shake itch itch itch shake itch itch` lit up over a real timeline is the single best screenshot in this project.
- The raw `PATTERN` and `DEFINE` clause displayed in a code block beside the visualisation. Show the SQL that found it.
- Sensitivity chart: match count per dog at loose / tuned / strict quantifiers.
- Syndrome frequency by breed group, small multiples.

Tab 5 — Baselines

- Per-dog: today versus own trailing baseline, as a band chart with the current line inside the shaded normal range.
- Breed cohort comparison: the selected dog against its cohort, z-scored.
- Small multiples grid, all dogs, deviation z-score, sorted worst first. A wall of sparklines where three are obviously wrong is a very effective image.

Tab 6 — Vet Note

- The Cortex SOAP note rendered as a clean clinical document, not a chat bubble.
- Triage badge at the top with the classification rationale.
- Evidence table beneath, linking each claim back to the matched epoch range.
- Model version and generation timestamp in the footer, because a note without provenance is not a clinical artefact.

Tab 7 — Drivers

- `TOP_INSIGHTS` output as a ranked contribution list: which dimension values move the deviation metric in surprising ways.
- Feature importance from the classification model, with `neck_back_corr` hopefully near the top.

- Confusion matrix heatmap from the held-out dogs, with the honest accuracy number printed large rather than hidden.

Tab 8 — Shelter Reality

- Austin intake reasons over time, behaviour-linked categories highlighted.
- Median length of stay by breed group and intake condition.
- The punchline chart described in section 12.
- One short paragraph of plain prose. This tab is allowed to be quiet.

Tab 9 — Pipeline

- DAG lineage rendered from `INFORMATION_SCHEMA`, with live refresh lag per node.
- Rows processed counter, epochs computed, syndrome scan latency.
- Task run history and model retrain log.
- Credit burn to date, which doubles as your own cost guard.
- The on-chain publish log with clickable Solscan links.

> **Tab order is a narrative.** Pack (who), Live Collar (is it real), Ethogram (what normal looks like), Syndromes (the finding), Baselines (why it is abnormal for *this* dog), Vet Note (what to do), Drivers (what explains it), Shelter Reality (why it matters), Pipeline (how it was built). A judge who clicks left to right gets the argument in order without reading a word of your post.

15. Visual design system

FERVOR used national colours and Geist type and it read as considered rather than default. Do the same thing here with a clinical register.

Element	Choice
Type	Geist or Inter throughout, tabular numerals for all metrics
Surface	Warm off-white `#FAFAF9`, cards `#FFFFFF` with a 1px `#E7E5E4` border, no drop shadows
Ink	`#1C1917` primary, `#57534E` secondary
Accent	Amber `#B45309`. One accent only. Everything else is greyscale plus status.
Triage	Routine `#15803D`, schedule `#B45309`, urgent `#B91C1C`
State palette	Rest and sit in cool greys, locomotion in a blue ramp, neck-dominant behaviours (shake, scratch) in amber so pathology literally stands out on the ribbon
Charts	No gridlines except a single horizontal baseline. No chart junk. Direct labelling instead of legends wherever a series can carry its own name.
Density	Clinical, not consumer. Small type, tight rows, numbers to a consistent precision. It should look like something a vet tech would use on a Tuesday.

One deliberate flourish, not three. The matched-pattern highlight on the Syndromes tab is where the design budget goes: coloured symbol blocks over a live timeline with the pattern string above it. Everywhere else, restraint.

16. Demo asset checklist

The FERVOR post carried nine images and that was a large part of why it won. Plan the images before you build, because half of them require the system to be in a particular state and you cannot recreate that at 3 AM on Monday.

#	File	What must be visible	When to capture
1	`01-cover.png`	Post cover: the pattern string over a state ribbon, TELLTAIL wordmark, tagline. Make this in Canva or Figma, not a screenshot.	Sunday night
2	`02-architecture.png`	The ten-stage round trip diagram from section 6, drawn cleanly	Anytime
3	`03-dag-lineage.png`	Snowsight lineage view of the real Dynamic Tables DAG. Proof the diagram is what Snowflake actually runs.	Once DAG is live
4	`04-schema-map.png`	Seven schemas, left to right lineage, drawn	Anytime
5	`05-syndrome-match.png`	**The hero image.** Syndromes tab, one match selected, matched epochs highlighted by symbol, pattern string above, SQL beside it.	After Gate F
6	`06-live-collar.png`	Raw 100 Hz waveform, neck and back, with the state ribbon underneath. Capture mid-replay so it looks alive.	During replay
7	`07-correlation-separation.png`	Box plot of neck/back correlation by true label. The feature justifying itself.	After Gate C
8	`08-ethogram-ribbon.png`	24-hour state ribbon plus the transition heatmap	After Gate D
9	`09-confusion-matrix.png`	Held-out dog confusion matrix with the honest accuracy number visible	After Gate D
10	`10-baseline-wall.png`	Small multiples grid of all dogs, three obviously deviating	After Gate F
11	`11-vet-note.png`	The Cortex SOAP note with triage badge and evidence table	After Gate G
12	`12-top-insights.png`	Drivers tab, ranked contributions, feature importance beside it	After Gate G
13	`13-sensitivity.png`	Match count per dog at loose / tuned / strict quantifiers	After Gate F
14	`14-shelter-punchline.png`	Syndrome categories against real Austin behaviour-linked outcomes	After Gate I
15	`15-solscan.png`	A confirmed devnet transaction on the explorer showing the TELLTAIL payload	After Gate I
16	`16-pipeline-tab.png`	Refresh lag, rows processed, task history, publish log	Sunday night

The video

Three to four minutes, screen recording, no talking head required.

1. **0:00–0:25** Cold open on the thesis. One sentence: a dog cannot tell you where it hurts. Show the Pack tab with one dog flagged red.
2. **0:25–1:00** Live Collar with the replayer running at `--speed 60`. Rows landing, waveform moving, states classifying. Prove it is live.
3. **1:00–2:10** Syndromes tab. Click the flagged dog. Show the match. Show the pattern string. Show the SQL. Say the line: this is a differential diagnosis written as a regular expression over rows.
4. **2:10–2:40** Vet Note and triage. The output a human can act on.
5. **2:40–3:10** Run `demo_spike.py` live, watch the anomaly detector fire, note the synthetic label on screen so nobody thinks you faked it.
6. **3:10–3:40** Shelter Reality, then the Solscan transaction. Close on the closing line.

> **Record the video on Sunday evening, not Monday morning.** Every hackathon post-mortem you have ever read says this and every hackathon entrant ignores it. The submission window closes at 12:29 IST Monday and a failed screen recording at 11:00 is how good projects lose.

17. Repository structure and run order

```
telltail/
  README.md
  HONESTY.md
  .env.example
  data/                          # gitignored; Kaggle CSVs land here
  scripts/
    run_sql.py                   # --all runs the whole warehouse in order
    load_raw.py                  # CSV -> Parquet -> stage -> COLLAR_TELEMETRY_BULK
    austin_sync.py               # Socrata -> REF.AAC_*
    demo_spike.py                # labelled synthetic deterioration, --clean to undo
    deploy_streamlit.py          # push app + environment.yml to stage
    profile_dataset.py           # Gate A: column and label profiling
  ingest/
    replay.py                    # bulk -> live micro-batches at wall-clock speed
  warehouse/
    00_account_setup.sql
    01_schemas.sql
    02_ref_seed.sql              # DOG_INFO, ETHOGRAM, SYNDROME_CATALOGUE, PARAMS
    03_raw.sql
    04_staging_dt.sql            # EPOCH_FEATURES dynamic table
    05_ml_classification.sql     # state model, eval, feature importance
    06_marts_dt.sql              # EPOCH_STATES, transitions, baselines
    07_syndromes.sql             # MATCH_RECOGNIZE views + task
    08_ml_timeseries.sql         # FORECAST, ANOMALY_DETECTION, TOP_INSIGHTS
    09_ai_layer.sql              # cached AI_COMPLETE / AI_CLASSIFY / AI_AGG
    10_oracle.sql                # PUBLISH_QUEUE, PUBLISH_LOG
    11_tasks.sql                 # every task, resumed in dependency order
    streamlit_app.py
    environment.yml
  bridge/
    server.ts                    # queue poll -> sign -> devnet -> audit back
  oracle-program/                # optional Anchor program, PDA per subject
  assets/                        # the sixteen images from section 16
```

Five-command quickstart for the README

```
git clone https://github.com/SoumyaEXE/telltail && cd telltail
cp .env.example .env                    # Snowflake creds, devnet keypair
pip install -r requirements.txt && npm install
python scripts/load_raw.py              # 10.6M rows -> Snowflake
python scripts/run_sql.py --all         # entire warehouse: schemas, DAG, models, tasks
python ingest/replay.py --speed 60      # the feed goes live
python scripts/deploy_streamlit.py      # dashboard inside the warehouse
npm run bridge                          # queue -> signed devnet writes
```

18. Known gotchas, pre-loaded

These are researched, not guessed. Half of them you already paid for during FERVOR. Put the ones you actually hit into the post as a war-stories table, because that table was one of the most useful things in your last write-up.

Symptom	Root cause	Fix
Dynamic table with `MATCH_RECOGNIZE` fails to create with `REFRESH_MODE = INCREMENTAL`	Unsupported construct for incremental refresh; `AUTO` silently resolves to `FULL` instead	Drive the syndrome layer with a task, or set `REFRESH_MODE = FULL` explicitly and document it
`MATCH_RECOGNIZE` query never returns	Pointed at the 100 Hz raw table, roughly 10.6M rows per partition scan	Epoch layer only. Aggregate to 1 second first.
Compilation error using `MATCH_RECOGNIZE` in a recursive CTE	Explicitly unsupported combination	Materialise the input first, then pattern match
Cortex anomaly detection rejects the model with an evaluation-timestamp error	Train and detect windows overlap	Split both at the same boundary, strict `<` on train, `>=` on detect
Classification accuracy suspiciously high	Random row split leaked adjacent samples from the same dog into train and test	Hold out entire dogs. Expect the honest number to be materially lower.
Every line chart in the dashboard is an identical straight diagonal	Snowpark `to_pandas()` returns numerics as object-dtype `Decimal`; Plotly treats them as categories so y becomes the row index	Convert element-wise with `float()` in the query helper and feed charts plain Python lists
`ModuleNotFoundError: plotly` in Streamlit in Snowflake only	Packages must be declared per app	Ship `environment.yml` next to the app file in the stage
`AttributeError: module 'streamlit' has no attribute 'column_config'`	SiS pins an older Streamlit than you develop against	`hasattr` guard; render tables as styled HTML
Scatter axes scale but zero points render	Old Plotly drops traces whose `customdata` is a mixed-type numpy array	Prebuild hover strings; pass `text=` plus `hoverinfo="text"`
Timestamps drift, epoch boundaries land wrong	Account timezone is not UTC	`ALTER ACCOUNT SET TIMEZONE = 'Etc/UTC';` before anything else
Cortex calls stop working mid-Sunday	Trial account AI Function daily credit cap exhausted	Batch and cache into tables; dedupe on the finding key; never call from a render path
External access integration rejected	Not supported on trial accounts	Outbound queue-poll bridge design, which is what you already built
SiS cannot fetch shelter data	Streamlit in Snowflake has no public internet egress	Host-side sync writes into `REF`; the app only reads a table
`COPY INTO` is slow or errors on the big CSV	Single 2 GB uncompressed file	Shard to Parquet, `PUT` with `AUTO_COMPRESS`, `MATCH_BY_COLUMN_NAME`

19. The honesty table

This goes in the post and in `HONESTY.md`. Judges reward the labelled compromise and punish the hidden one, and you already know this because it worked last time.

Simplification	Why	How it is labelled
Telemetry is replayed from a published research dataset, not streamed from a live collar	No collar hardware in a weekend; the dataset is real, video-annotated, peer-reviewed data from 45 dogs	`is_replay = TRUE` on every landed row, stated plainly in the post
Syndrome patterns are clinically motivated, not clinically validated	Validation requires veterinary study design and ethics approval, not a weekend	Explicit statement: this is not a diagnostic device. Patterns cite their behavioural rationale, not a trial.
Some ethogram states may be heuristic rather than model-derived	The dataset's core protocol covers locomotion and posture; fine-grained health behaviours may not be labelled	`state_source` column, values `MODEL` or `HEURISTIC`, surfaced in the UI
Demo deterioration is injected	A recorded demo needs the detector to fire on camera	`is_synthetic = TRUE`; detection sees it, training never fits it, `--clean` removes it
Attestations go to devnet; the bridge is centralised	This is an attestation bridge, not a decentralised oracle network	Stated plainly, as before
Shelter data is Austin only	One city publishes a decade of clean, current, public intake and outcome records	Named in the tab and the post; not generalised to national claims
Classification accuracy is reported on held-out dogs and is lower than row-split figures elsewhere	Row splits leak; dog-disjoint splits are the honest protocol for this dataset	The number is printed large on the Drivers tab rather than buried

> **Two sentences that cost nothing and buy a lot.** One: TELLTAIL is not a diagnostic device and nothing it produces substitutes for a veterinarian. Two: full credit to Vehkaoja, Somppi, Törnqvist, Valldeoriola Cardó, Kumpulainen, Väätäjä, Majaranta, Surakka, Kujala, Vainio et al., University of Helsinki, whose dataset makes this possible.

20. Hour-by-hour timeline

Deadline: Monday 17 August 2026, 12:29 PM IST. Everything below is IST.

When	Target	Gate
Fri evening	Snowflake account, Kaggle download, the two smoke tests (MATCH_RECOGNIZE, Cortex). Nothing else. If either smoke test fails you need Saturday to work around it.	—
Sat 09:00–10:30	Profile the dataset. Real column names, real label vocabulary, per-dog counts. Decide the ethogram.	A
Sat 10:30–12:30	Schemas, bulk load 10.6M rows, replayer working, rows visibly landing	B
Sat 12:30–14:30	Epoch feature Dynamic Table. Validate the neck/back correlation separates labels. Capture image 07.	C
Sat 14:30–17:00	Classification, dog-disjoint split, confusion matrix, states populated. Capture images 08 and 09.	D
Sat 17:00–18:30	**One syndrome matching on real data.** The riskiest thing in the build. Do not proceed until it works.	E
Sat 19:30–23:30	The full syndrome catalogue plus the sensitivity sweep. This is the creative work; give it real hours and do not rush it.	F
Sun 09:00–11:00	Baselines, FORECAST, ANOMALY_DETECTION, TOP_INSIGHTS	F
Sun 11:00–12:30	Cortex layer, batched into tables, budget checked	G
Sun 12:30–18:00	Streamlit app, tab by tab, in narrative order. Capture screenshots as each tab lands rather than at the end.	H
Sun 18:00–20:00	Austin sync, Solana bridge, README, HONESTY.md	I
Sun 20:00–21:30	**Record the video.** Non-negotiable slot.	—
Sun 21:30–01:00	Write the post. Cover image. Push the repo with a clean history.	—
Mon 09:00–11:30	Buffer. Something will be broken and this is where you fix it.	—
Mon 11:30	Submit. One hour before the deadline, not five minutes.	—

> **The rule that saves the weekend:** if Gate E has not passed by Saturday 18:30, stop adding syndromes and ship one working syndrome with a great write-up. One pattern that provably fires on real data, explained beautifully, beats six patterns that half work.

21. The DEV post outline

Title candidates

1. We Taught a Snowflake Warehouse to Diagnose a Dog From the Way It Moves
2. A Dog Cannot Tell You Where It Hurts. Ten Million Rows Can.
3. Your Dog Tracker Counts Steps. We Wrote the Differential Diagnosis as a Regex Over Rows.

Option 1 mirrors your winning title's shape, which is a real advantage with the same judging pool. Option 3 is the most technically arresting. Pick 1 for continuity or 3 for impact.

Structure

Section	Job
Cold open	The mistake everyone makes: activity trackers compute totals because that is what `AVG()` gives you. But a vet diagnoses from a sequence. Land the thesis in three sentences, as you did with the "database and a logo" line last time.
The result first	Hero image 05: a real syndrome match on a real dog, pattern lit up over the timeline. Show the finding before the architecture.
The architecture, actually explained	The ten-stage table. Every stage does load-bearing work; there is no component whose only job is to name-drop a technology. Reuse that exact framing, it worked.
The data is real	45 dogs, 27 breeds, 100 Hz dual IMU, video-annotated, peer-reviewed, 10.6 million rows. Credit the authors.
The feature that earned its place	Neck/back correlation. One `CORR()`, physically interpretable, and the box plot proving it separates the classes.
The syndrome catalogue	The creative payload. Six patterns, the table of why-sequence-not-threshold, the SQL, the sensitivity sweep.
The split nobody else did	Dog-disjoint folds and why row splits lie. Print the honest accuracy.
Cortex, clinically	SOAP notes, triage, pack brief, all batched. Contrast the register with a consumer app deliberately.
The shelter tab	The emotional close, kept short and quiet.
The attestation	Why a chain, in one paragraph, with the keypair-never-touches-the-warehouse property stated for judges to verify.
War stories	The gotchas you actually hit, in a table. Readers loved this last time.
What's real vs what's simplified	Section 19 verbatim.
The bigger thing	Strip away the dogs and the pattern is: row pattern recognition turns a warehouse into a diagnostic engine for any sensored system where the sequence carries the meaning. Fleet telemetry, industrial equipment, patient monitoring. The plumbing is exactly what is in this repo.
Run it yourself	The five commands
Video	Embedded

The closing line

The dog was showing the pattern for eleven days before anyone noticed. The warehouse noticed on day two.

22. Judging criteria map

Criterion	How TELLTAIL answers it
Relevance to theme	The premise only exists because dogs cannot speak. The theme is load-bearing, not applied. Real dog data, real dog problem, real dog outcomes.
Creativity	Nobody expects a differential diagnosis to be a regex over rows. The syndrome catalogue is an original artefact, and the neck/back correlation feature is a genuine insight from reading the dataset properly.
Technical execution	10.6M rows, a declarative Dynamic Tables DAG, supervised classification with an honest holdout protocol, row pattern recognition, three ML functions, an LLM layer, a live dashboard inside the warehouse, and a signed on-chain attestation. All running, all documented.
Writing quality	Thesis-first structure, result before architecture, war stories, an explicit honesty table. The formula that won last time, applied to a stronger premise.
Use of Snowflake	`MATCH_RECOGNIZE`, Dynamic Tables with declared lag, `ASOF JOIN`, `ML.CLASSIFICATION`, `FORECAST`, `ANOMALY_DETECTION`, `TOP_INSIGHTS`, `AI_COMPLETE`, `AI_CLASSIFY`, `AI_AGG`, Streamlit in Snowflake. `MATCH_RECOGNIZE` and `TOP_INSIGHTS` in particular are features almost nobody reaches for in a weekend.
Use of Solana	A defensible reason for a chain (portability across institutions that share no database) rather than a logo, with a checkable security property.

23. Sources and verification notes

Everything technical in this document was verified against primary documentation before it was written down. Where a claim is a preview feature or account-dependent, that is noted.

Claim	Source	Status
`MATCH_RECOGNIZE` syntax, quantifiers, `MEASURES`, `AFTER MATCH SKIP`, `ONE ROW PER MATCH`	Snowflake docs, SQL reference and the row-pattern-matching user guide	Generally available
Cannot be used inside a recursive CTE	Snowflake docs, MATCH_RECOGNIZE introduction	Explicit limitation
Explicit `REFRESH_MODE = INCREMENTAL` fails on unsupported constructs; `AUTO` resolves to `FULL`	Snowflake docs, supported queries for dynamic tables	Confirmed
ML functions: Forecasting, Anomaly Detection, Classification, Top Insights	Snowflake ML functions overview	GA / preview varies by function and account
AISQL multimodal across `AI_COMPLETE`, `AI_TRANSCRIBE`, `AI_CLASSIFY`, `AI_EMBED`, `AI_SIMILARITY`, processing staged files	Snowflake Cortex AI Functions multimodal docs	Mostly GA; audio and video via `AI_COMPLETE` in public preview
Trial accounts without a payment method are capped at roughly ten credits per day of Cortex AI Function usage	Snowflake trial accounts docs	Confirmed, and the single biggest schedule risk
External access integrations unsupported on trial accounts	Your own FERVOR build, error 509009	Confirmed the hard way
Dataset: 45 dogs, 27 breeds, 100 Hz, dual sensor (neck collar + back harness), 6-DOF, video-annotated at one-second resolution, 10,611,068 rows across 20 columns	Vehkaoja et al., *Data in Brief* 2022; Mendeley Data; Kaggle mirror	Confirmed
Single-subject classifiers drop from roughly 91% to 70–74% when generalising to unseen dogs	Canine behaviour classification literature, cited in the dataset paper's related work	Confirmed; the reason for dog-disjoint folds
Austin Animal Center intakes and outcomes, October 2013 onward, over 90% adopted / transferred / returned, behaviour named among outcome reasons	City of Austin open data portal, Socrata	Confirmed, updated continuously
Labelled bark corpora with context annotation exist (stranger, play, isolation, distress)	Published bioacoustic datasets and Kaggle mirrors	Available; treat as stretch scope only

One last thing. The reason FERVOR won was not Snowflake and it was not Solana. It was that you asked what each platform could do that the other physically cannot, answered it honestly, and then built exactly that with nothing decorative attached. The question this time is: what can a warehouse understand about a body that the body cannot say out loud. Keep answering that question and the rest is execution.